

CS151 Intro to Data Structures

Arrays, Generics, Nested Classes

Announcements

- HW0 due Friday
- Lab02 today - due Tues Sep 15th
 - autograded
- HW1 is released
 - Due next Friday Sep 18th
 - You can start it after lab today

Outline

- Review: Arrays
- Generics
- Nested Classes

Warm up 1

- Order from least to most expensive:
 - insert at beginning
 - insert at end
 - remove from end
- Runtime Analysis of each operation (answer in Big-O Notation):
 - Accessing an element
 - Inserting an element
 - Removing an element

Arrays Review

- Java arrays: Sequential, contiguous, memory layout
 - gives us $O(1)$ access (if we know the index)
- fixed size
- `insert(E elem, int idx)`
- `remove(int idx)`
- `set(E elem, int idx)`

Warm up 2

[A, B, C, D, _, _, _, _] numElems = 4, size = 8

Trace each operation in order. Draw the array, and update numElems after each step.

insert(X, 2)

remove(0)

insert(Y, 4)

remove(2)

Java.util.ArrayList

- <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>
- `import java.util.ArrayList`
- `ExpandableArray` is a simple `ArrayList`

Methods of an ArrayList

<code>add(o)</code>	appends o at the end of list
<code>add(index, o)</code>	inserting given o at index, shifting list to the right
<code>get(index)</code>	returns the object found at index
<code>remove(index)</code>	removes the object found at index and returns it, shifting list to the left
<code>set(index, o)</code>	replaces object at given index with o
<code>size()</code>	returns the number of elements in list
<code>indexOf(o)</code>	returns the first index where o is found, or -1
<code>lastIndexOf(o)</code>	returns the last index where o is found, or -1
<code>clear()</code>	removes all

Warm up 3

Add a method `max()` to `ExpandableArray` that returns the largest element in the array.

Warm up 4

Add a method `isSorted()` to `ExpandableArray` that returns `true` if the elements are in ascending order (each element $\leq$ the next one), and `false` otherwise.

Generics

ExpandableArray

We created an ExpandableArray for int types

```
private int[] data; //underlying array
```

- What if we want an ExpandableArray for doubles, Strings, Students, etc.
- We could create an ExpandableArray for each type... but now this violates our goal of **reusability**
- This brings us to **Generics**

Generics

- First, let's look at some code.
- A way to write classes or methods that can operate on a variety of data types without being locked into specific types at the time of definition
- Write definitions with type parameters

```
public <T> void print(T x) {  
    System.out.println(x);  
}
```

Generic Classes

- We just implemented a generic print method
- Let's see how to make our `ExpandableArray` generic
 - **Code!**

Generics Arrays

Can not create arrays of parameterized types!

```
private T[] array = new T[10]; is not valid
```

- Casting to the rescue!

- `T[] array = (T[]) new Object[10];`

Other Generic Restrictions

Can not declare static instance variables of a parameterized type

```
private static T MAX_SIZE;
```

```
//compiler error: non-static type variable T  
cannot be referenced from a static context
```

Generic Static Methods

These are allowed!

```
//Static class method  
public static <T> void getMax(T t) {  
    System.out.println(t);  
}
```

```
//Class instance Method:  
public T get(int index) {  
    return this.data[index];  
}
```

Nested Classes

- A class defined inside the definition of another class
- Benefits:
 - Encapsulation (data hiding and access control)

Nested Classes

- An instance of the inner class can't be created without an instance of the outer class.
- **Code**

Nested Classes - Access modifiers

- An inner class can access **all** members of the outer class

```
Person.this.name;
```

- An outer class can access **all** members in the inner class
- Even when they're private!

Summary

- **HW0 due Friday**
- Java arrays are *fixed-size*, contiguous, and sequential
- We made our ExpandableArray generic
 - What does this mean?
- Nested Classes